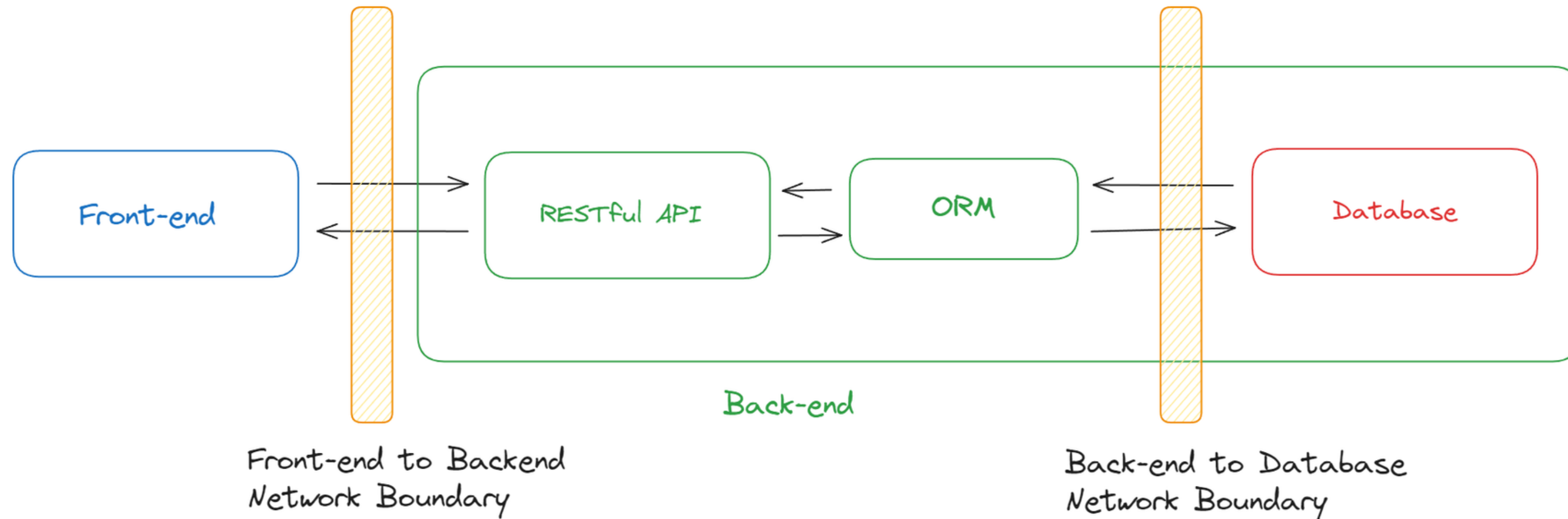


The architecture of a sample full-stack project



You should have a good idea about client server architecture

The Front-end

The Front-end

- **The user facing part of a software system.**
- **Different ways to develop front-ends.**
 - a. HTML, CSS, Javascript
 - b. Using a front-end library/framework
 - i. Component based front-end libraries
 - 1. ReactJS, VueJS, Svelte, Solid
 - ii. HATEOAS based front-end libraries
 - 1. AlpineJS
 - 2. HTMX

The most in demand front-end library for internship positions is ReactJS

ReactJS

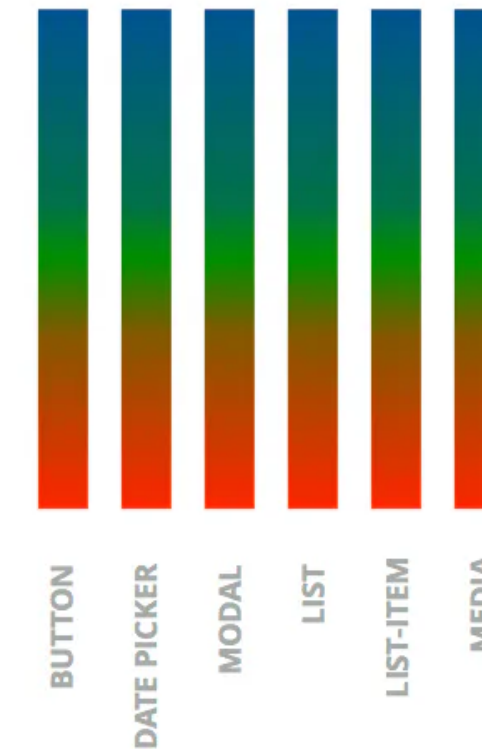
- **Key characteristics**
 - Declarative
 - Component based separation of concerns
 - Reduces cross-file dependencies
 - Less prone to errors
 - Uses the concept of a UI as a function of application state
 - V-DOM
 - Rendering and re-rendering behavior
 - Using React Hooks

Separation of Concerns



Separation of Concerns

(only, from a different point of view)



You should also have an idea about

- Using a package manager confidently,
 - NPM, Yarn
- Basic terminal commands
 - cd, ls

Core elements of a ReactJS based Web Application

Build Tool

Router

Styling Solution

**Form Management
Solution**

State management

**Async state
management**

Ideally, you should be able to explain what are the solutions, or approaches to that you chose tackle each of these and why you chose them.



Build Tools

- Vite
- Create React App



Router

- React Router
- Tanstack React Router



Styling Solutions

- Plain CSS
- CSS Modules
- TailwindCSS
- CSS-IN-JS & Styled Components

State Management Patterns

- React rendering, re-rendering behavior
- Basic state via useState, It's limitations, Prop drilling
- Global state management solutions
 - Redux
 - Zustand



Form Management Solutions

- React Hook Form
- Formik



Async State Management Solutions

- RTK Query
- Tanstack React Query
- SWR

The Back-end

The Back-end

- The part where the service logic of your application takes place.
- Have a general idea about different back-end frameworks and their tradeoffs.
 - a. NodeJS/ExpressJS
 - b. NestJS
 - c..Net Core
 - d. Spring Boot
- You should have experience in at least one back-end framework .
- We recommend Node/Express as it has the lowest barrier to entry as an intern.
- The tradeoff with choosing a more structured framework is then you are exposed to a larger pool of concepts and practices that could be asked during the interview

Monolithic vs Microservice based back-ends

- A Monolithic back-end is where **all** the service logic required for the operation of a web application is built as a **single** web service.
- A Microservices based back-end is where the service logic of the application is separated into multiple web services and designed to work while interacting with each other.
- As an intern having experience with building a Monolithic back-end is enough..

NodeJS/ExpressJS

- NodeJS is a runtime. An environment where Javascript can be run outside the browser.
- ExpressJS is the backend framework that is built on top of NodeJS to let developers build their back-end web services in a declarative manner.

Core elements of a back-end web service

RESTful API

ORM

You should be able to explain what are the solutions, or approaches to that you chose tackle each of these and why you chose them.



Database

Database

- The part where your application data is persisted.
- Types of databases
 - a. Relational Databases
 - b. Document Databases
 - c. Graph Databases
- **You should have experience in at least one type of databases.**
- **You should also have a general understanding and tradeoffs of other types of databases as well.**

Front-end Back-end Network Boundary

Front-end Back-end Network Boundary

- If we are having a full-stack project where we have a separate front-end and a back-end, the front-end needs to communicate with the back-end across a network.

Back-end Database Network Boundary

Back-end Database Network Boundary

- Usually the database runs on a separate dedicated database server and the server where our backend has to connect to it via specific credentials